

Ejercicio 1

Define una clase **PilaEnteros** que gestione una pila de enteros, almacenándolos internamente como un array de enteros. La idea es ir almacenando los elementos consecutivamente en el array, de manera que el último elemento guardado en el array es considerado la cima de la pila.

La pila se crea con una capacidad máxima. Define la clase *PilaEnteros* con al menos un constructor que acepte como argumento el tamaño máximo que tendrá la pila, un método **void apilar(int)** que permita apilar un entero y otro método **int cima()** que devuelva la cima de la pila (o sea, el último entero apilado).

En esta clase se pueden producir dos tipos de errores:

1. Que intentemos apilar más elementos de los que caben en la pila.
2. Que queramos obtener la cima de la pila cuando está vacía.

Queremos que la pila controle su desbordamiento al intentar apilar más elementos de los que admite mediante una excepción de usuario **ExcepcionDesbordamiento**. Por otro lado, el método *cima()* deberá lanzar una excepción de usuario, llamada **ExcepcionPilaVacía** cuando se le invoque sobre una pila vacía.

Implementa un programa principal de prueba que compruebe el correcto funcionamiento de la clase, capturando las excepciones cuando

1. intenta obtener la cima de una pila vacía
2. intenta apilar más elementos de los permitidos. Asegúrate de que se capture la excepción en cuanto se produzca, dejando de intentar apilar el resto de elementos que sobran.

Ejercicio 2

Indica qué se mostrará por pantalla cuando se ejecute cada una de estas clases:

```
class Uno{
    private static int metodo(){
        int valor=0;
        try{
            valor = valor +1;
            valor = valor + Integer.parseInt("42");
            valor = valor + 1;
            System.out.println("Valor al final del try: " + valor);
        }catch (NumberFormatException e){
            valor = valor + Integer.parseInt("42");
            System.out.println("Valor al final del catch: " + valor);
        }finally{
            valor = valor + 1;
            System.out.println("Valor al final de finally: " + valor);
        }
        valor = valor + 1;
        System.out.println("Valor antes del return: " + valor);
        return valor;
    }

    public static void main(String[] args){
        try{
            System.out.println(metodo());
        }catch (Exception e){
            System.err.println("Excepcion en metodo()");
            e.printStackTrace();
        }
    }
}
```

```
class Dos{
    private static int metodo(){
        int valor=0;
        try{
            valor = valor +1;
            valor = valor + Integer.parseInt("W");
            valor = valor + 1;
            System.out.println("Valor al final del try: " + valor);
        }catch (NumberFormatException e){
            valor = valor + Integer.parseInt("42");
            System.out.println("Valor al final del catch: " + valor);
        }finally{
            valor = valor + 1;
            System.out.println("Valor al final de finally: " + valor);
        }
        valor = valor + 1;
        System.out.println("Valor antes del return: " + valor);
        return valor;
    }

    public static void main(String[] args){
        try{
            System.out.println(metodo());
        }catch (Exception e){
            System.err.println("Excepcion en metodo()");
            e.printStackTrace();
        }
    }
}
```

```

class Tres{
    private static int metodo(){
        int valor=0;
        try{
            valor = valor +1;
            valor = valor + Integer.parseInt("W");
            valor = valor + 1;
            System.out.println("Valor al final del try: " + valor);
        }catch(NumberFormatException e){
            valor = valor + Integer.parseInt("W");
            System.out.println("Valor al final del catch: " + valor);
        }finally{
            valor = valor + 1;
            System.out.println("Valor al final de finally: " + valor);
        }
        valor = valor + 1;
        System.out.println("Valor antes del return: " + valor);
        return valor;
    }

    public static void main(String[] args){
        try{
            System.out.println(metodo());
        }catch(Exception e){
            System.err.println("Excepcion en metodo()");
            e.printStackTrace();
        }
    }
}

```

```

import java.io.*;

class Cuatro{
    private static int metodo(){
        int valor=0;
        try{
            valor = valor +1;
            valor = valor + Integer.parseInt("W");
            valor = valor + 1;
            System.out.println("Valor al final del try: " + valor);
            throw new IOException();
        }catch(IOException e){
            valor = valor + Integer.parseInt("42");
            System.out.println("Valor al final del catch: " + valor);
        }finally{
            valor = valor + 1;
            System.out.println("Valor al final de finally: " + valor);
        }
        valor = valor + 1;
        System.out.println("Valor antes del return: " + valor);
        return valor;
    }

    public static void main(String[] args){
        try{
            System.out.println(metodo());
        }catch(Exception e){
            System.err.println("Excepcion en metodo()");
            e.printStackTrace();
        }
    }
}

```

Ejercicio 3

Ingresar los nombres y las notas de n alumnos y reportar la lista en orden alfabético y en orden de mérito.

Ejercicio 4

Se tiene la siguiente información de n personas: dni, apellidos, nombres, sexo, edad y peso. Esta información se tiene que registrar en un ArrayList, buscar una persona dado el dni, eliminar una persona, ordenar por apellidos y mostrar todas las personas. Hacer un menú para realizar lo que se pide.

Ejercicio 5

Crear un programa que use ArrayList de números reales. El programa debe permitir mostrar un menú donde se pueda agregar un número, buscar un número, modificar un número, eliminar un número e insertar un número en una posición dada